0124. Binary Tree Maximum Path Sum

| Category: Trees | LeetCode: https://leetcode.com/problems/binary-tree-maxim

Problem

- A path is any sequence of connected nodes (each used once, not necessarily through the root). Return the maximum path sum. Values may be negative.

Intuition

- Same shape as the diameter problem: a best path bends at some node, using the best downward "arm" from each side.
- An arm with a negative sum should be dropped (take 0 instead).
- Return to the parent only one arm (node + the better child arm), since a path cannot fork.

Brute Force

- Try every pair of nodes as endpoints: O(n^2) or worse.

Optimal Approach

- Gain(node): null → 0.
- L = max(0, gain(left)), r = max(0, gain(right)).
- Best = max(best, node.val + l + r).
- Return node.val + max(l, r).

Worked Example

```
[-10,9,20,null,null,15,7].

gain(15)=15 gain(7)=7
at 20: best = 20+15+7 = 42 ; return 20+15 = 35
gain(9)=9
at -10: best = max(42, -10+9+35=34) = 42
answer 42
```

Java Solution

```java
public class BinaryTreeMaximumPathSum {
    private int best;

    public int maxPathSum(TreeNode root) {
        best = Integer.MIN_VALUE;
        gain(root);
        return best;
    }

    private int gain(TreeNode n) {
        if (n == null) return 0;
        int l = Math.max(0, gain(n.left)), r = Math.max(0, gain(n.right));
        best = Math.max(best, n.val + l + r);
        return n.val + Math.max(l, r);
    }
}
```

Complexity

- Time: O(n). Space: O(h).

Pitfalls

- Initialise best to the minimum (or root value), not 0: an all-negative tree's answer is negative.